

INSTITUTO MAUÁ DE TECNOLOGIA
ENGENHARIA DE COMPUTAÇÃO

RELATÓRIO TÉCNICO — PROJETO DE MICROCONTROLADORES E SISTEMAS
EMBARCADOS

RADVEM — Radar Didático com Varredura Eletromecânica

Disciplina: EEN251- Microcontroladores e Sistemas Embarcados
Professores(as): Sergio Ribeiro Augusto e Rodrigo de Marca Franca
Integrantes: Gabriel Barrochelo, Ruth Ramos Romeu e Vinícius Eloy Araujo

São Caetano do Sul – SP
2026

Gabriel Barrochelo
Ruth Ramos Romeu
Vinícius Eloy Araujo

RADVEM — Radar Didático com Varredura Eletromecânica

Relatório técnico apresentado à disciplina EEN251 – Microcontroladores e Sistemas Embarcados como requisito parcial de avaliação do curso de Engenharia da Computação do Instituto Mauá de Tecnologia.

Professores: Sergio Ribeiro Augusto e Rodrigo de Marca Franca

São Caetano do Sul – SP
2026

RESUMO

O presente trabalho apresenta o desenvolvimento do RADVEM — Radar Didático com Varredura Eletromecânica, um sistema embarcado destinado à demonstração prática de conceitos relacionados a microcontroladores, sensores, atuadores e integração hardware–software. O projeto foi desenvolvido utilizando um microcontrolador Raspberry Pi Pico, um sensor ultrassônico HC-SR04, um servo motor SG90, um display OLED SSD1306 e um sistema de alimentação híbrido em 3,3 V e 5 V com conversão de nível lógico. O sistema realiza uma varredura angular automatizada por meio de um servo motor, executando medições contínuas de distância em tempo real. A lógica implementada permite interromper automaticamente o movimento diante da presença de obstáculos próximos, retomando a operação quando a condição de bloqueio deixa de existir. O desenvolvimento empregou MicroPython, controle de versões via Git/GitHub, testes incrementais de hardware e software e técnicas de depuração aplicadas ao ambiente embarcado. Como resultado, foi obtido um protótipo funcional, modular e didático, capaz de integrar sensores, atuadores, interface gráfica e controle lógico em uma solução coesa voltada ao ensino de sistemas embarcados.

Palavras-chave: sistemas embarcados; microcontroladores; Raspberry Pi Pico; sensor ultrassônico; servo motor; MicroPython.

SUMÁRIO

RESUMO	3
1. INTRODUÇÃO	5
2. METODOLOGIA.....	6
2.1 Levantamento de requisitos	6
2.2 Escopo do projeto	7
2.3 Tecnologias empregadas.....	8
2.4 Desenvolvimento, testes e depuração	10
2.5 Design mecânico e análise preliminar de custos	11
3. RESULTADOS E ANÁLISE DOS DADOS	12
3.1 Integração hardware–software.....	12
3.2 Funcionamento do sistema	13
3.3 Lógica de detecção de obstáculos.....	14
3.4 Interface OLED e monitoramento em tempo real	15
3.5 Diagrama de blocos do sistema	15
3.7 Discussão dos resultados	16
4. CONCLUSÃO.....	17
REFERÊNCIAS	19
APÊNDICE A – DESCRIÇÃO DO SOFTWARE DESENVOLVIDO	19
APÊNDICE B – HISTÓRICO DE VERSIONAMENTO.....	22
ANEXO A – DIAGRAMA DE BLOCOS	23
ANEXO B – REGISTRO FOTOGRÁFICO	23

1. INTRODUÇÃO

A crescente presença de sistemas embarcados em aplicações industriais, automotivas, domésticas e educacionais torna essencial o domínio de conceitos associados à integração entre sensores, atuadores, programação embarcada e eletrônica aplicada. Nesse contexto, plataformas de prototipagem rápida baseadas em microcontroladores têm assumido papel relevante no ensino de engenharia, permitindo o desenvolvimento de soluções experimentais com elevada flexibilidade de implementação.

O presente projeto propõe o desenvolvimento do RADVEM — Radar Didático com Varredura Eletromecânica, um sistema embarcado projetado para demonstrar princípios fundamentais de sensoriamento espacial, controle de movimento e interface homem-máquina utilizando componentes eletrônicos de baixo custo e ampla disponibilidade acadêmica.

A solução desenvolvida utiliza o microcontrolador Raspberry Pi Pico como unidade central de processamento, responsável pelo controle de um servo motor SG90, pela aquisição de dados provenientes de um sensor ultrassônico HC-SR04 e pela atualização visual das informações em um display OLED SSD1306. O sistema realiza uma varredura angular contínua do ambiente, associando medições de distância à posição instantânea do mecanismo eletromecânico.

Além do desenvolvimento funcional do protótipo, o trabalho contempla aspectos relacionados ao levantamento de requisitos, definição de escopo, seleção tecnológica, implementação incremental, depuração de software, documentação técnica e controle de versões utilizando Git e GitHub, em conformidade com os objetivos da disciplina EEN251 – Microcontroladores e Sistemas Embarcados.

O projeto também incorpora considerações práticas relacionadas à compatibilidade elétrica entre dispositivos de diferentes níveis de tensão, à organização da montagem física e ao planejamento de uma estrutura mecânica adequada para acomodação dos componentes.

O objetivo geral do trabalho consiste em desenvolver um sistema embarcado capaz de realizar detecção de obstáculos por meio de varredura eletromecânica automatizada, integrando sensores, atuadores, interface gráfica e controle lógico.

Como objetivos específicos, destacam-se:

- a) implementar um mecanismo de varredura angular controlado por servo motor;
- b) realizar medições de distância em tempo real utilizando tecnologia ultrassônica;
- c) desenvolver uma interface visual baseada em display OLED para apresentação das informações do sistema;
- d) implementar mecanismos de controle lógico para bloqueio e retomada automática do movimento mediante detecção de obstáculos;
- e) integrar hardware e software em um protótipo funcional documentado por meio de ferramentas de versionamento e práticas de engenharia de desenvolvimento.

2. METODOLOGIA

2.1 Levantamento de requisitos

O desenvolvimento do RADVEM iniciou-se com a identificação dos requisitos necessários para a implementação de um sistema embarcado capaz de demonstrar princípios de detecção espacial e controle eletromecânico. O levantamento foi dividido entre requisitos funcionais e requisitos não funcionais, permitindo uma definição clara das capacidades esperadas do protótipo.

Requisitos funcionais

RF01 — Controle de ativação do sistema

O sistema deveria permitir a ativação e desativação do funcionamento por meio de um botão físico externo conectado ao microcontrolador.

RF02 — Varredura angular automatizada

O sistema deveria executar movimentação contínua do servo motor em uma faixa angular predeterminada, realizando deslocamento de ida e volta sem intervenção do usuário.

RF03 — Medição contínua de distância

O sensor ultrassônico deveria executar leituras periódicas do ambiente durante toda a operação do sistema.

RF04 — Detecção automática de obstáculos

O sistema deveria identificar objetos próximos através de limiares de distância configurados em software.

RF05 — Interrupção automática do movimento

Ao detectar obstáculos dentro da zona de segurança estabelecida, o sistema deveria interromper o movimento do servo motor.

RF06 — Retomada automática da operação

Após o afastamento do obstáculo detectado, o sistema deveria retornar automaticamente ao funcionamento normal.

RF07 — Interface visual em tempo real

O display OLED deveria apresentar continuamente o estado operacional do sistema, o ângulo instantâneo do servo motor e a distância medida.

Requisitos não funcionais

RNF01 — Compatibilidade elétrica

O projeto deveria operar respeitando os limites elétricos do Raspberry Pi Pico, exigindo tratamento adequado das diferenças de tensão entre componentes.

RNF02 — Robustez de software

O sistema deveria evitar travamentos decorrentes de falhas de leitura do sensor ultrassônico ou instabilidades na comunicação entre dispositivos.

RNF03 — Modularidade do código

A implementação deveria manter separação lógica entre aquisição de dados, controle do servo motor, interface visual e gerenciamento do estado do sistema.

RNF04 — Baixo custo de implementação

Os componentes selecionados deveriam possuir ampla disponibilidade e custo compatível com aplicações didáticas e acadêmicas.

RNF05 — Reprodutibilidade experimental

O sistema deveria permitir replicação simples em ambiente de laboratório.

2.2 Escopo do projeto

O escopo do RADVEM compreende o desenvolvimento de um protótipo funcional de radar didático baseado em varredura eletromecânica utilizando microcontrolador, sensores e atuadores integrados.

O projeto contempla:

- controle lógico via Raspberry Pi Pico;
- movimentação angular automatizada por servo motor;
- aquisição de distância utilizando sensor ultrassônico;

- interface visual baseada em display OLED;
- implementação de lógica de bloqueio por obstáculo;
- integração elétrica entre dispositivos operando em diferentes níveis de tensão;
- desenvolvimento incremental utilizando controle de versões;
- planejamento de montagem física e design mecânico.

Por outro lado, algumas funcionalidades permaneceram fora do escopo da solução desenvolvida, visando preservar a viabilidade técnica e temporal do projeto.

Não fazem parte do escopo:

- comunicação sem fio;
- processamento de imagens;
- armazenamento persistente de dados;
- geração gráfica bidimensional semelhante a radares comerciais;
- integração com aplicações web ou dispositivos móveis.

A delimitação do escopo permitiu direcionar o esforço de desenvolvimento para os objetivos centrais da disciplina, concentrando a implementação em conceitos de sistemas embarcados, integração hardware–software e controle de dispositivos físicos.

2.3 Tecnologias empregadas

O desenvolvimento do RADVEM demandou a utilização integrada de tecnologias de hardware, software, documentação técnica e ferramentas de engenharia de desenvolvimento.

Hardware empregado

O sistema foi implementado utilizando os seguintes componentes eletrônicos:

Raspberry Pi Pico (RP2040)

Atuando como unidade central de processamento, o microcontrolador executa a lógica principal do sistema, realizando leitura dos dispositivos de entrada, geração de sinais PWM, comunicação I2C e gerenciamento dos estados operacionais.

Sensor ultrassônico HC-SR04

Responsável pela medição de distância através da emissão e recepção de pulsos ultrassônicos.

Servo motor SG90

Utilizado para executar a movimentação angular do sistema de varredura.

Display OLED SSD1306 (I2C)

Empregado como interface gráfica embarcada para exibição de informações em tempo real.

Conversor de nível lógico 3,3 V ↔ 5 V

Necessário para compatibilizar eletricamente o sinal ECHO do sensor ultrassônico com as entradas do Raspberry Pi Pico.

Botão industrial C/NA/NF

Utilizado para implementação do controle de ativação do sistema.

Fonte externa de alimentação

Empregada para alimentar adequadamente o servo motor, evitando sobrecarga elétrica no microcontrolador.

Software empregado

O software embarcado foi desenvolvido utilizando **MicroPython**, devido à sua simplicidade de implementação, rapidez de prototipagem e ampla adoção em ambientes educacionais.

O desenvolvimento foi realizado na IDE **Thonny**, utilizada para transferência de código, execução de testes e monitoramento serial.

As principais bibliotecas empregadas foram:

- machine
- time
- ssd1306

A biblioteca ssd1306.py foi integrada ao projeto para gerenciamento do display OLED por meio de comunicação I2C.

Controle de versão e documentação

Durante o desenvolvimento foi utilizado **GitHub** como plataforma de versionamento, documentação incremental e rastreamento de alterações do software.

O histórico de commits registra etapas relevantes do processo de desenvolvimento, incluindo:

- implementação funcional do protótipo;
- refinamento da documentação do projeto;
- reorganização da estrutura de arquivos;

- ajuste dos limites angulares do servo motor;
- calibração dos limiares de detecção de distância.

Entre os commits registrados destacam-se:

- feat(RADVEM): Protótipo v1.0.1 funcional;
- feat(RADVEM): Protótipo v1.0.1 finalizado;
- Modify servo angle limits and distance thresholds;
- Rename botao.py to main.py;
- atualizações estruturais do arquivo README.

O uso de GitHub contribuiu para organização do desenvolvimento, rastreabilidade das modificações e documentação do ciclo evolutivo do projeto.

2.4 Desenvolvimento, testes e depuração

O desenvolvimento do RADVEM ocorreu por meio de uma estratégia incremental baseada em testes unitários de hardware seguidos pela integração gradual dos módulos.

Inicialmente foram realizados testes isolados dos componentes individuais, permitindo validação preliminar do funcionamento de cada dispositivo.

O botão foi testado através da leitura contínua de níveis lógicos utilizando entradas digitais configuradas com resistor interno *pull-down*.

O display OLED foi validado mediante testes de comunicação I2C, incluindo varredura de endereços utilizando a função `i2c.scan()`.

O servo motor passou por etapas sucessivas de calibração relacionadas à frequência PWM, largura de pulso e limites angulares de movimentação.

Durante o desenvolvimento observou-se que alterações inadequadas nos parâmetros de temporização poderiam modificar simultaneamente velocidade e posicionamento do servo, exigindo refinamento da função responsável pela conversão angular.

O sensor ultrassônico foi submetido a testes específicos de temporização utilizando medições baseadas em `ticks_us()`. Para evitar travamentos decorrentes de ausência de resposta do sensor, foi implementado mecanismo de *timeout* no processo de leitura.

Outro desafio técnico identificado esteve relacionado à incompatibilidade elétrica entre o sinal ECHO do HC-SR04 e o nível lógico suportado pelo Raspberry Pi Pico. A solução adotada consistiu na utilização de um conversor de nível lógico dedicado.

A depuração do software foi conduzida utilizando:

- monitor serial (print());
- testes incrementais;
- isolamento funcional dos módulos;
- validação de GPIOs;
- análise de temporização;
- ajustes iterativos dos parâmetros de controle.

Essa abordagem permitiu identificar falhas progressivamente, reduzindo a complexidade da integração final.

2.5 Design mecânico e análise preliminar de custos

Além do desenvolvimento eletrônico e computacional, o projeto incluiu planejamento preliminar da estrutura mecânica destinada à montagem final do protótipo.

O conceito mecânico prevê uma caixa compacta capaz de acomodar:

- Raspberry Pi Pico;
- módulo OLED;
- sensor HC-SR04;
- servo motor;
- conversor lógico;
- sistema de alimentação;
- cabeamento interno organizado.

O sensor ultrassônico é montado mecanicamente sobre o eixo do servo motor, permitindo que o conjunto execute a varredura angular do ambiente.

O projeto também considera a realização de soldagem permanente das conexões em placa dedicada, substituindo o estágio inicial baseado em protoboard.

Foi planejada organização interna orientada à redução de interferência elétrica, separando linhas de alimentação do servo motor dos sinais de comunicação do sistema.

Uma análise preliminar de custos foi realizada considerando preços médios de mercado dos componentes utilizados.

Componente	Quantidade	Custo
Raspberry Pi Pico	1	R\$ 50,00
HC-SR04	1	R\$ 15,00
Servo SG90	1	R\$ 5,00
OLED SSD1306	1	R\$ 20,00
Conversor lógico	1	R\$ 15,00
Botão C/NA/NF	1	R\$ 10,00
Fonte externa 5V	1	R\$ 50,00
Fios jumper	diversos	R\$ 5,00
Placa de soldagem	1	R\$ 5,00
Caixa mecânica	1	R\$ 20,00
Total		R\$ 195,00

O custo total estimado do sistema situa-se em aproximadamente **R\$ 195,00**, demonstrando viabilidade econômica para aplicações acadêmicas e educacionais.

3. RESULTADOS E ANÁLISE DOS DADOS

3.1 Integração hardware–software

A implementação final do RADVEM resultou em um sistema embarcado funcional capaz de integrar sensores, atuadores, interface visual e lógica de controle utilizando o microcontrolador Raspberry Pi Pico.

A arquitetura desenvolvida baseou-se na comunicação coordenada entre múltiplos dispositivos periféricos conectados às interfaces GPIO do microcontrolador.

A distribuição final das conexões físicas é apresentada a seguir.

Componente	Pino Raspberry Pi Pico	Função
Botão	GPIO 1	Entrada digital
OLED SDA	GPIO 4	Comunicação I2C
OLED SCL	GPIO 5	Comunicação I2C
HC-SR04 TRIG	GPIO 2	Saída digital
HC-SR04 ECHO	GPIO 3	Entrada digital
Servo SG90	GPIO 15	PWM
Alimentação lógica	3,3 V	Raspberry Pi Pico
Alimentação potência	5 V externo	Servo motor

Durante a integração observou-se a necessidade de tratamento das diferenças de tensão entre dispositivos.

O módulo HC-SR04 opera tipicamente em 5 V, enquanto o Raspberry Pi Pico utiliza lógica de 3,3 V. Dessa forma, foi incorporado um conversor de nível lógico entre o terminal **ECHO** do sensor e a entrada digital do microcontrolador.

Além disso, devido à limitação de corrente disponível no regulador interno do Pico, adotou-se alimentação externa dedicada para o servo motor SG90.

Essa separação reduziu instabilidades elétricas associadas a quedas de tensão e flutuações de corrente durante acelerações do atuador.

O sistema final demonstrou operação estável após integração completa dos subsistemas.

3.2 Funcionamento do sistema

O comportamento operacional do RADVEM foi implementado por meio de uma máquina lógica baseada em estados de ativação, detecção de obstáculos e controle de movimento angular.

O ciclo de funcionamento inicia-se a partir da interação do usuário com o botão físico externo.

A lógica implementada utiliza configuração *toggle*, alternando entre estado ativo e estado inativo a cada acionamento detectado.

Para minimizar leituras espúrias provenientes de bouncing mecânico, foi implementado um mecanismo simples de *debounce* baseado em temporização.

Quando o sistema encontra-se ativo, o servo motor inicia a varredura angular automática.

O movimento ocorre dentro do intervalo definido entre **0°** e **150°**, realizando deslocamento contínuo de ida e volta.

O controle direcional foi implementado por meio das variáveis *angulo* e *direcao*.

A cada ciclo de execução:

- o ângulo é atualizado;
- a posição do servo é recalculada;
- uma nova medição de distância é realizada;
- a interface gráfica é atualizada.

A função responsável pelo controle do servo converte valores angulares em sinais PWM apropriados para o SG90.

O algoritmo implementado calcula a largura de pulso necessária utilizando interpolação linear entre pulsos mínimos e máximos.

Durante os testes verificou-se que pequenas alterações nos parâmetros PWM poderiam modificar simultaneamente velocidade e posicionamento do motor, exigindo calibração experimental dos limites adotados.

A leitura de distância é realizada utilizando o sensor HC-SR04.

O algoritmo implementado executa as seguintes etapas:

1. emissão de pulso de disparo (*trigger*);
2. aguardo da ativação do sinal **ECHO**;
3. medição temporal do pulso recebido;
4. cálculo da distância utilizando velocidade do som.

Para maior robustez operacional, foram implementados mecanismos de *timeout* capazes de interromper leituras inválidas.

Essa estratégia evita bloqueios do programa decorrentes de ausência de retorno do sensor.

3.3 Lógica de detecção de obstáculos

Uma das funcionalidades centrais do RADVEM consiste na interrupção automática do movimento diante da aproximação de objetos.

O comportamento foi implementado utilizando lógica baseada em histerese.

A histerese foi escolhida para eliminar oscilações rápidas de estado próximas ao limite de detecção.

O algoritmo utiliza dois limiares distintos:

Condição	Distância
-----------------	------------------

Bloqueio do movimento	$\leq 12 \text{ cm}$
-----------------------	----------------------

Liberação do movimento	$\geq 15 \text{ cm}$
------------------------	----------------------

Quando a distância medida torna-se inferior ou igual a 12 cm, a variável de estado bloqueado assume valor verdadeiro.

Nesse cenário, o servo motor interrompe imediatamente sua movimentação.

Quando o obstáculo afasta-se novamente, ultrapassando 15 cm, o sistema remove automaticamente o bloqueio e retoma a operação normal.

A implementação dessa estratégia mostrou-se eficaz na eliminação do comportamento conhecido como *tremedeira*, identificado durante os testes preliminares.

Sem histerese, pequenas variações de leitura próximas ao limiar provocavam sucessivas ativações e desativações do servo motor.

Com a solução implementada, observou-se estabilidade significativamente superior.

3.4 Interface OLED e monitoramento em tempo real

O display OLED SSD1306 foi utilizado como interface embarcada de acompanhamento operacional do sistema.

A comunicação foi implementada utilizando protocolo I2C.

O display apresenta continuamente informações relevantes para operação e depuração do sistema.

As informações exibidas incluem:

- estado operacional do sistema;
- ângulo instantâneo do servo;
- distância medida;
- indicador de bloqueio.

Quando o sistema está ativo, o display apresenta o status **Servo: ON**.

No estado inativo, são exibidas mensagens de orientação ao usuário.

Quando a lógica de bloqueio é acionada, surge na interface o indicador textual **STOP**, permitindo rápida identificação visual da condição de obstáculo detectado.

A atualização contínua da interface contribuiu tanto para validação experimental quanto para depuração do software.

Durante o desenvolvimento, o OLED foi frequentemente utilizado como mecanismo de observabilidade interna do sistema.

3.5 Diagrama de blocos do sistema

O funcionamento global do RADVEM pode ser representado pelo diagrama de blocos mostrado na Figura 1.

Radar ultrassônico com servo motor

- compatibilidade elétrica entre dispositivos;
- alimentação de cargas eletromecânicas;
- sincronização temporal;
- tratamento de falhas de leitura.

A utilização de testes incrementais demonstrou importância significativa na redução da complexidade de integração.

A adoção de mecanismos de *timeout* e histerese aumentou substancialmente a robustez do sistema.

Outro fator relevante foi a utilização de GitHub como ferramenta de organização do desenvolvimento.

O histórico de versionamento permitiu registrar sucessivas melhorias do software, incluindo refinamentos na lógica de controle, documentação das conexões físicas e reorganização estrutural do projeto.

O sistema final apresentou desempenho compatível com os objetivos didáticos propostos, demonstrando adequadamente conceitos fundamentais de:

- sistemas embarcados;
- integração hardware–software;
- controle de atuadores;
- aquisição de sensores;
- desenvolvimento incremental.

Como possibilidades futuras de expansão, destacam-se:

- geração gráfica semelhante a radares reais;
- comunicação serial com computador;
- visualização em interface gráfica externa;
- armazenamento de medições;
- implementação de comunicação sem fio.

4. CONCLUSÃO

O desenvolvimento do RADVEM – Radar Didático com Varredura Eletromecânica permitiu a aplicação prática dos conceitos estudados na disciplina de Microcontroladores e Sistemas

Embarcados, abrangendo desde o levantamento de requisitos até a integração final entre hardware e software.

O projeto atingiu os objetivos inicialmente propostos ao implementar um sistema capaz de realizar varredura angular automatizada, medição de distância em tempo real, detecção de obstáculos e exibição das informações em uma interface visual embarcada. A utilização do Raspberry Pi Pico como unidade de processamento demonstrou-se adequada para o gerenciamento simultâneo dos dispositivos conectados, proporcionando desempenho satisfatório para a aplicação desenvolvida.

Durante o desenvolvimento foram enfrentados desafios técnicos relacionados à alimentação do servo motor, compatibilidade entre níveis lógicos, comunicação I2C com o display OLED e tratamento de falhas na leitura do sensor ultrassônico. A solução desses problemas exigiu análise de documentação técnica, realização de testes incrementais e aplicação de técnicas de depuração, contribuindo para o amadurecimento da equipe em práticas de desenvolvimento de sistemas embarcados.

A implementação da lógica de histerese para controle da detecção de obstáculos mostrou-se fundamental para garantir estabilidade operacional, eliminando oscilações indesejadas próximas ao limite de detecção. Da mesma forma, a utilização de mecanismos de timeout aumentou a robustez do software ao evitar bloqueios decorrentes de leituras inválidas do sensor.

O uso de Git e GitHub possibilitou o controle estruturado das versões do projeto, permitindo registrar a evolução do desenvolvimento, documentar alterações e manter rastreabilidade sobre as funcionalidades implementadas. Essa prática contribuiu significativamente para a organização do trabalho e para a manutenção da qualidade do software produzido.

Sob a perspectiva do design mecânico, foi elaborado o planejamento de uma estrutura física capaz de acomodar os componentes eletrônicos, garantir proteção ao sistema e permitir a movimentação adequada do conjunto formado pelo servo motor e sensor ultrassônico. A proposta busca tornar o protótipo mais robusto, portátil e adequado para demonstrações acadêmicas.

O projeto também evidenciou a importância do trabalho colaborativo entre os integrantes da equipe, envolvendo divisão de tarefas, compartilhamento de conhecimento e resolução conjunta dos problemas encontrados durante o desenvolvimento.

Como trabalhos futuros, destacam-se a implementação de representação gráfica semelhante a radares comerciais, comunicação com computadores para visualização avançada dos dados, armazenamento histórico das medições e integração com tecnologias de comunicação sem fio.

Dessa forma, conclui-se que o RADVEM atendeu aos objetivos propostos e constituiu uma experiência relevante para a consolidação dos conhecimentos relacionados a sistemas embarcados, sensores, atuadores, programação em MicroPython e integração hardware–software.

REFERÊNCIAS

Raspberry Pi Ltd.. *Raspberry Pi Pico Documentation*. Disponível em: <https://www.raspberrypi.com/documentation/microcontrollers/>. Acesso em: 03 jun. 2026.

MicroPython. *MicroPython Documentation*. Disponível em: <https://docs.micropython.org/>. Acesso em: 03 jun. 2026.

HC-SR04 Ultrasonic Sensor. *HC-SR04 Ultrasonic Sensor Datasheet*. Disponível em documentação técnica do fabricante. Acesso em: 03 jun. 2026.

SG90 Micro Servo. *SG90 Servo Motor Datasheet*. Disponível em documentação técnica do fabricante. Acesso em: 03 jun. 2026.

SSD1306 OLED Display. *SSD1306 OLED Controller Datasheet*. Disponível em documentação técnica do fabricante. Acesso em: 03 jun. 2026.

[GitHub Documentation](#). Acesso em: 03 jun. 2026.

APÊNDICE A – DESCRIÇÃO DO SOFTWARE DESENVOLVIDO

O software principal do projeto foi implementado em MicroPython e estruturado em dois arquivos principais:

main.py

```
from machine import Pin, I2C, PWM
import time
import ssd1306

# Botão
botao = Pin(1, Pin.IN, Pin.PULL_DOWN)

# OLED
i2c = I2C(0, scl=Pin(5), sda=Pin(4))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

# Servo
servo = PWM(Pin(15))
servo.freq(50)

# Ultrassônico
trig = Pin(2, Pin.OUT)
echo = Pin(3, Pin.IN)

def medir_distancia():
    trig.low()
    time.sleep_us(2)
    trig.high()
    time.sleep_us(10)
    trig.low()

    inicio = 0
    fim = 0
    timeout = time.ticks_us()

    while echo.value() == 0:
        inicio = time.ticks_us()
        if time.ticks_diff(time.ticks_us(), timeout) > 30000:
            return None
```

```

        timeout = time.time()
        while echo.value() == 1:
            fim = time.time()
            if time.time() - timeout > 30000:
                return None

        duracao = time.time() - inicio
        return (duracao * 0.0343) / 2

def set_angle(angle):
    min_us = 500
    max_us = 2500
    pulse = min_us + (angle / 180) * (max_us - min_us)
    duty = int(pulse / 20000 * 65535)
    servo.duty_u16(duty)

# Controle
ativo = False
ultimo_estado = 0

# Controle do movimento
angulo = 0
direcao = 1 # 1 = indo, -1 = voltando

# Controle de obstáculo
bloqueado = False

while True:
    # --- BOTÃO ---
    estado = botao.value()

    if estado == 1 and ultimo_estado == 0:
        ativo = not ativo
        time.sleep(0.2)

    ultimo_estado = estado

    # --- SENSOR ---
    distancia = medir_distancia()

    # Histerese (evita tremedeira)
    if distancia is not None:
        if distancia <= 12:
            bloqueado = True
        elif distancia >= 15:
            bloqueado = False

    # --- SERVO ---
    if ativo and not bloqueado:
        set_angle(angulo)

        # Atualiza ângulo (varredura)
        angulo += 5 * direcao

        if angulo >= 150:
            angulo = 150
            direcao = -1
        elif angulo <= 0:
            angulo = 0
            direcao = 1

        time.sleep(0.05)

    else:
        servo.duty_u16(0)

    # --- OLED ---
    oled.fill(0)

    if ativo:
        oled.text("Servo: ON", 0, 0)
    else:
        oled.text("Servo: OFF", 0, 0)
        oled.text("Aperte o botao", 0, 10)

    oled.text("Ang:", 0, 20)
    oled.text(str(angulo), 40, 20)

```

```

oled.text("Dist:", 0, 40)

if distancia is not None:
    oled.text(str(round(distancia, 1)), 50, 40)
else:
    oled.text("--", 50, 40)

if bloqueado:
    oled.text("STOP", 80, 0)

oled.show()

time.sleep(0.01)

```

ssd1306.py

MicroPython SSD1306 OLED driver, I2C and SPI interfaces

```

from micropython import const
import framebuffer

SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xa4)
SET_NORM_INV = const(0xa6)
SET_DISP = const(0xae)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xa0)
SET_MUX_RATIO = const(0xa8)
SET_COM_OUT_DIR = const(0xc0)
SET_DISP_OFFSET = const(0xd3)
SET_COM_PIN_CFG = const(0xda)
SET_DISP_CLK_DIV = const(0xd5)
SET_PRECHARGE = const(0xd9)
SET_VCOM_DESEL = const(0xdb)
SET_CHARGE_PUMP = const(0x8d)

class SSD1306:
    def __init__(self, width, height, external_vcc):
        self.width = width
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        self.framebuf = framebuffer.FrameBuffer(self.buffer, self.width, self.height,
        framebuffer.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP | 0x00,
            SET_MEM_ADDR, 0x00,
            SET_DISP_START_LINE | 0x00,
            SET_SEG_REMAP | 0x01,
            SET_MUX_RATIO, self.height - 1,
            SET_COM_OUT_DIR | 0x08,
            SET_DISP_OFFSET, 0x00,
            SET_COM_PIN_CFG, 0x12,
            SET_DISP_CLK_DIV, 0x80,
            SET_PRECHARGE, 0x22,
            SET_VCOM_DESEL, 0x30,
            SET_CONTRAST, 0xff,
            SET_ENTIRE_ON,
            SET_NORM_INV,
            SET_CHARGE_PUMP, 0x14,
            SET_DISP | 0x01):
            self.write_cmd(cmd)
        self.fill(0)
        self.show()

    def fill(self, col):
        self.framebuf.fill(col)

    def text(self, string, x, y, col=1):
        self.framebuf.text(string, x, y, col)

```

```

def show(self):
    for page in range(self.pages):
        self.write_cmd(0xb0 + page)
        self.write_cmd(0x00)
        self.write_cmd(0x10)
        self.write_data(self.buffer[self.width * page:self.width * (page + 1)])

class SSD1306_I2C(SSD1306):
    def __init__(self, width, height, i2c, addr=0x3C):
        self.i2c = i2c
        self.addr = addr
        self.temp = bytearray(2)
        super().__init__(width, height, False)

    def write_cmd(self, cmd):
        self.temp[0] = 0x80
        self.temp[1] = cmd
        self.i2c.writeto(self.addr, self.temp)

    def write_data(self, buf):
        self.i2c.writeto(self.addr, b'\x40' + buf)

```

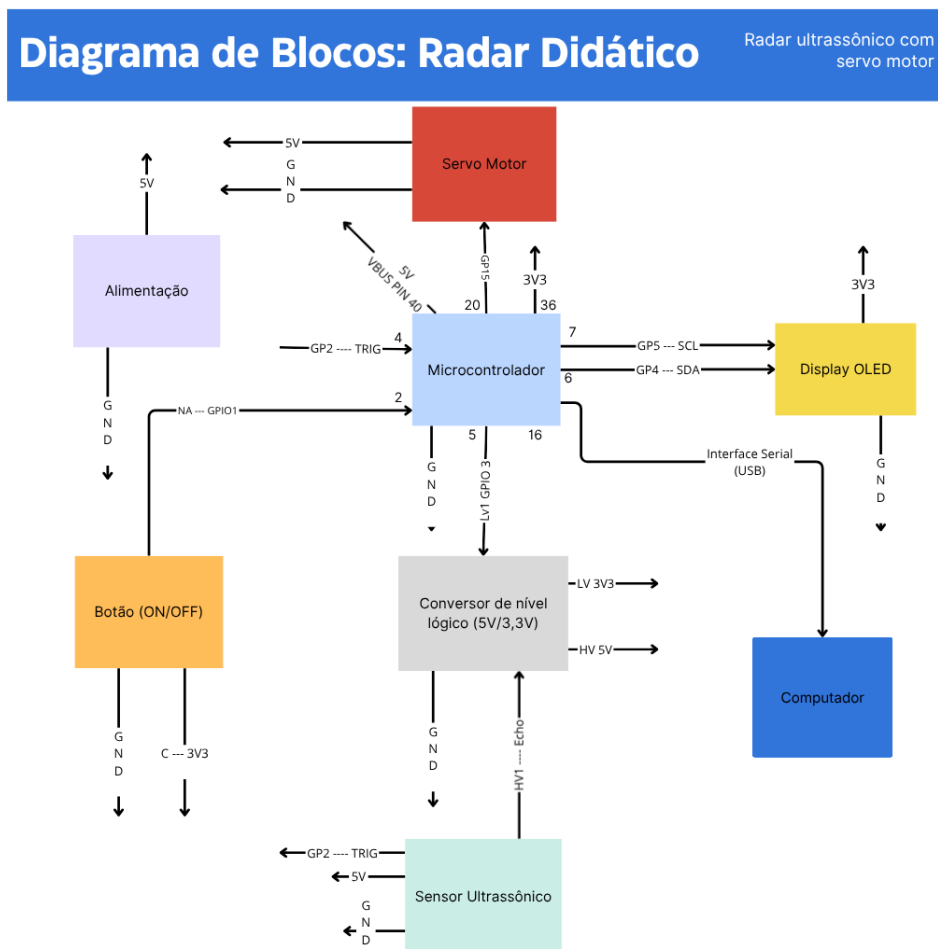
APÊNDICE B – HISTÓRICO DE VERSIONAMENTO

Durante o desenvolvimento foram registrados os seguintes marcos de evolução do software:

Commit	Descrição
feat(RADVEM): Protótipo v1.0.1 finalizado	Finalização da primeira versão funcional
feat(RADVEM): Protótipo v1.0.1 funcional	Integração dos módulos principais
Expansão do README	Inclusão das conexões físicas e documentação
Rename botao.py to main.py	Reestruturação do código
Modify servo angle limits and distance thresholds	Ajustes de calibração
Format README.md with proper headings and structure	Padronização da documentação

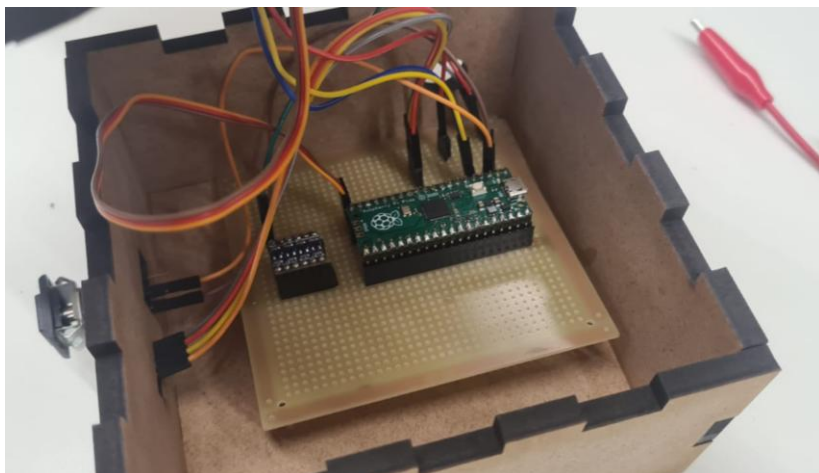
O versionamento permitiu acompanhar a evolução incremental do projeto e registrar as decisões tomadas durante o desenvolvimento.

ANEXO A – DIAGRAMA DE BLOCOS

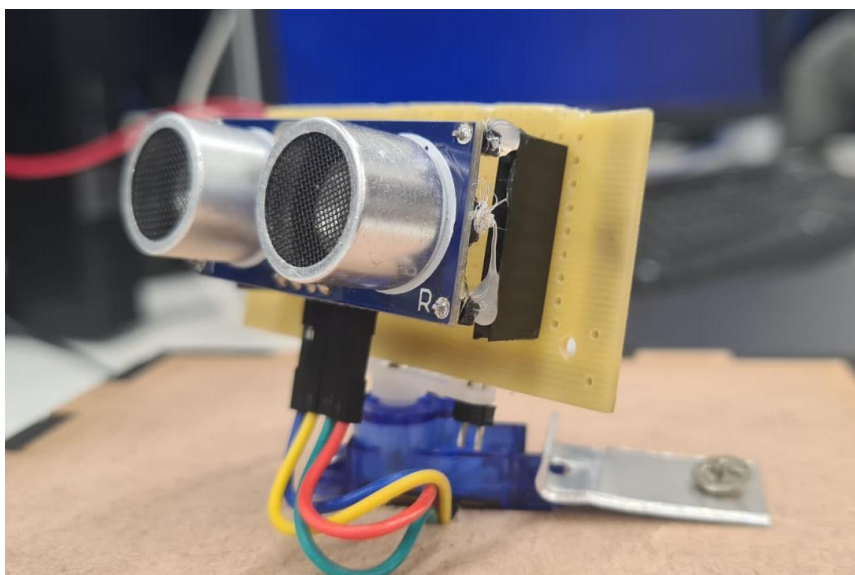


Legenda: Diagrama de Blocos do Radar Didático

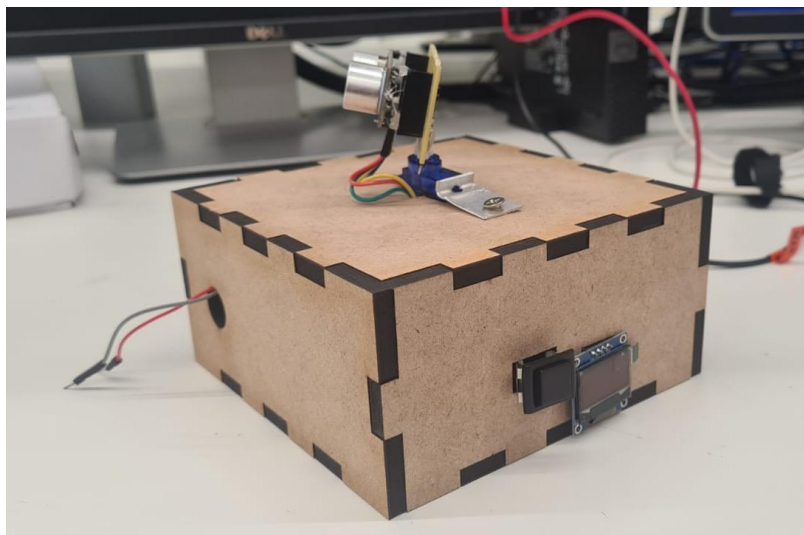
ANEXO B – REGISTRO FOTOGRÁFICO



Legenda: Interior da caixa.



Legenda: Sensor ultrassônico montado sobre o servo-motor.



Legenda: Caixa montada com todos os componentes.



Legenda: Acabamento do botão.



Legenda: Integrantes do grupo com o projeto finalizado.